# Jawaban Technical Test — AI-Driven Fullstack Developer

## Halaman Judul

| Data | Keterangan |
|---|---|
| **Nama** | Ahmad Isha Latif Qolbi |
| **Posisi** | Fullstack Developer (Paid Internship) — PT Tirtamas Coldstorindo Logistik |
| **Tanggal** | 26 October 2026 |
| **Email** | ishalatif15@gmail.com |
| **Telepon** | 085704027719 |
| **Repository** | https://github.com/ishalatif/Warehouse-Receiving |
| **Link PDF** | https://drive.google.com/drive/folders/110HWBjy_SDIrbufH_QnkNjL8mxxSp83z?usp=sharing |

**Judul Proyek:** Warehouse Receiving System — aplikasi penerimaan barang gudang dengan backend Express/TypeScript dan frontend Next.js.

---

## 1. Ringkasan Eksekutif

Saya membangun aplikasi web untuk mencatat penerimaan barang gudang. Supervisor membuat katalog produk dan dokumen penerimaan; Operator mencatat kuantitas yang benar-benar diterima. Setiap penerimaan memperbarui item dokumen, saldo stok, ledger transaksi, dan status dokumen dalam **satu transaksi database**, sehingga tidak ada stok yang bertambah tanpa catatan.

Fokus implementasi bukan hanya fitur, tetapi **integritas data**: over-receiving ditolak, dua request paralel tidak membuat stok ganda, dan kegagalan di tengah proses mengembalikan seluruh perubahan (rollback). Aturan ini ditegakkan di backend dan diuji terhadap PostgreSQL nyata, bukan hanya dengan mock.

**Hasil verifikasi terakhir (26 Oktober 2026):**

| Pemeriksaan | Hasil |
|---|---|
| Build backend + frontend | Lulus |
| Unit + integration backend | **93/93** lulus (16 file) |
| Unit test frontend | **22/22** lulus |
| Audit dependency produksi | **0 kerentanan** |
| Deployment Docker Compose | Clean start, migrasi sekali, readiness `200 → 503 → 200` |

---

## 2. Ruang Lingkup

### Termasuk

- Login JWT dan RBAC Supervisor/Operator.
- Katalog produk dengan SKU unik.
- Pembuatan, daftar, detail, dan pembatalan dokumen penerimaan ( `DRAFT` ).
- Penerimaan parsial/penuh, proteksi over-receiving.
- Stok dan riwayat transaksi.
- Readiness check yang memeriksa PostgreSQL.
- Docker Compose untuk PostgreSQL, migrasi, backend, dan frontend.

### Tidak termasuk (v1)

- Barcode, multi-gudang, batch/lot, tanggal kedaluwarsa, supplier, retur, koreksi ledger.
- Pagination/filter dan idempotency key.
- Registrasi publik atau manajemen user berbasis UI.

---

## 3. Arsitektur dan Keputusan Teknis

```
Browser (Next.js 16)
        │  HTTP + JWT
        ▼
Express API (TypeScript)
  controller → service → PostgreSQL
        │
        ▼
PostgreSQL 15 (constraint + transaksi)
```

| Keputusan | Alasan |
|---|---|
| Backend sebagai sumber kebenaran | Validasi dan invariant tidak bergantung pada UI |
| Service layer memegang transaksi | Satu unit kerja: item, stok, ledger, status |
| `SELECT ... FOR UPDATE` | Mencegah dua penerimaan paralel membaca sisa yang sama |
| Constraint database | `received_quantity <= expected_quantity` , stok `>= 0` , SKU unik ternormalisasi |
| Ledger append-only | Riwayat stok dapat diaudit dan direkonsiliasi |
| JWT + RBAC middleware | Pemisahan tegas hak Supervisor dan Operator |
| Migrasi dengan advisory lock | Aman jika dijalankan lebih dari sekali |
| Docker Compose + healthcheck | Urutan start: DB sehat → migrasi selesai → backend sehat → frontend |

### Alur penerimaan (atomik)

1. Validasi payload dan role Operator.
2. `BEGIN`.
3. Lock dokumen (`FOR UPDATE`), lalu lock seluruh item dengan urutan ID konsisten.
4. Validasi status dokumen, produk, duplikasi, dan sisa kuantitas **sebelum menulis**.
5. Update `receiving_items`.
6. Upsert `inventory`.
7. Insert `inventory_transactions`.
8. Update status dokumen (`IN_PROGRESS` / `COMPLETED`).
9. `COMMIT`; error apa pun → `ROLLBACK` seluruh langkah.

---

## 4. Pemenuhan Kebutuhan

| Kebutuhan | Implementasi | Bukti |
|---|---|---|
| Login dan token | `POST /api/auth/login`, bcrypt, JWT 1 jam | Test route + HTTP login nyata |
| RBAC | Middleware `authenticate` + `requireRole` | Operator → `403` pada produk/dokumen; Supervisor → `403` pada receive |
| Produk & SKU unik | Unique index `lower(btrim(sku))` | Integration test duplikat kapital/spasi |
| Dokumen `DRAFT` | Dibuat beserta item, stok tidak berubah | Integration test |
| Penerimaan parsial/penuh | Update received, stok, ledger, status | Test parsial → `IN_PROGRESS`, penuh → `COMPLETED` |
| Over-receiving | Ditolak sebelum menulis | Test rollback multi-item |
| Concurrency | Lock dokumen dan item | Dua request paralel: hanya satu berhasil |
| Rollback kegagalan tulis | Transaksi + fault injection | TC-17: gagal insert ledger dan gagal update status |
| Stok & ledger konsisten | Saldo = agregat ledger | Test rekonsiliasi |
| Health | Query PostgreSQL nyata | `200` siap, `503` saat DB mati, `200` setelah pulih |
| Deployment | Compose, migrasi sekali, healthcheck | Clean start terisolasi |

---

## 5. Keamanan

- Password disimpan sebagai hash bcrypt (cost 12); respons login tidak pernah memuat hash.
- Seluruh query memakai parameter terikat; tidak ada interpolasi input ke SQL.
- Endpoint terproteksi menolak token hilang, rusak, kedaluwarsa, atau role tidak valid.
- Pesan error ke klien tidak memuat stack trace/detail internal.

- Tidak ada akun default, seed, atau kredensial di repository.
- Akun lokal dibuat satu per satu melalui CLI; password tidak di-echo dan tidak lewat argumen shell.

---

## 6. Cara Menjalankan

```
# 1. Siapkan environment
Copy-Item .env.example .env
# isi POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB, JWT_SECRET, CORS_ORIGIN

# 2. Jalankan stack
docker compose up -d --build --wait

# 3. Cek readiness
curl http://localhost:5000/api/health

# 4. Buat akun (sekali per user)
docker compose exec -it -e NODE_ENV=development backend node dist/scripts/create-user.js
```

- UI: `http://localhost:3000`
- API: `http://localhost:5000/api`
- PostgreSQL tidak dipublikasikan ke host.

### Menjalankan test

```
cd backend
npm test          # integration butuh TEST_DATABASE_URL disposable
npm run build

cd ../frontend
npm test
npm run build
```

---

## 7. Hasil Pengujian

| Perintah / skenario | Hasil |
|---|---|
| `backend npm run build` | PASS — `tsc` exit 0 |
| `backend npm test` + `TEST_DATABASE_URL` | PASS — 93/93, 16 file |
| `frontend npm test` | PASS — 22/22, 3 file |
| `frontend npm run build` | PASS — Next.js 16.4.3 |

| Perintah / skenario | Hasil |
|---|---|
| `npm audit --omit=dev` (frontend) | PASS — 0 kerentanan |
| Compose clean start | PASS — semua service healthy, migrate `Exited (0)` |
| Migrasi diulang | PASS — tidak menerapkan ulang |
| Readiness DB siap → mati → pulih | PASS — `200` → `503` → `200`, backend tidak crash |
| Login nyata + HTTP → database | PASS — token valid, RBAC, stok/ledger sesuai |
| TC-17 rollback kegagalan tulis | PASS — item, stok, ledger, status kembali konsisten |
| UI E2E lintas role (user persisten) | Belum dijalankan — dicatat sebagai gap, bukan klaim lulus |

**Isolasi pengujian:** seluruh test yang menulis data memakai PostgreSQL disposable ( `--tmpfs` , loopback) yang dihapus setelah sesi. Tidak ada database pengguna yang disentuh.

---

## 8. Keterbatasan dan Rekomendasi

1. **UI E2E dengan user persisten** belum dijalankan; langkah berikutnya adalah membuat dua akun lokal dan menjalankan skenario Supervisor/Operator di browser.
2. **Rebuild image Docker** setelah upgrade Next.js belum diverifikasi karena registry tidak dapat dijangkau saat sesi terakhir; build lokal dan artefak standalone sudah lulus.
3. **Audit dev-only:** 3 temuan moderate pada toolchain Tailwind 3; perbaikan memerlukan Tailwind 4 (major) dan direncanakan terpisah.
4. Untuk produksi: rotasi `JWT_SECRET` , TLS, rate limiting login, backup PostgreSQL, dan observability belum termasuk v1.

---

## 9. Lampiran

- Repository: https://github.com/ishalatif/Warehouse-Receiving
- Dokumentasi proyek: `docs/`
- Riwayat pengembangan: `docs/riwayat_pengembangan.md`
- Rencana dan bukti QA: `docs/rencana_pengujian_qa.md`
- Spesifikasi API: `docs/openapi_specification.md`
- Panduan prompting AI: `docs/panduan_prompting.md`

---